



MODUL 9 • DATABASE DENGAN PRISMA 7

Migrations: migrate dev vs migrate deploy



Manage perubahan database schema dengan aman — versioned, reviewable, reversible.

prisma migrate dev — Untuk Development

Buat dan jalankan migration baru. Gunakan hanya di local, tidak di production.

Workflow development dengan migrate dev

```
# 1. Edit schema.prisma (tambah/ubah model)

# 2. Jalankan migrate dev
bunx prisma migrate dev --name add_post_table
# ↑ --name = nama migration yang deskriptif

# Yang terjadi:
# ✓ Buat file migration SQL di prisma/migrations/
# ✓ Jalankan migration ke database lokal
# ✓ Generate ulang Prisma Client
# ✓ Jalankan seed jika ada (konfigurasi di prisma.config.ts)

# Contoh file yang dibuat:
# prisma/migrations/
# 20241201120000_add_post_table/
# migration.sql

# Isi migration.sql (auto-generated):
# CREATE TABLE "posts" (
#   "id" TEXT NOT NULL,
#   "title" TEXT NOT NULL,
#   -- ...
# );

# 3. Commit migration file ke git!
git add prisma/migrations
git commit -m "feat: add post table migration"
```

Kapan Pakai migrate dev?

- ✓ Tambah model baru
- ✓ Tambah/ubah field
- ✓ Tambah/hapus relasi
- ✓ Ubah constraint atau index

- ✗ Jangan di production!
- ✗ Jangan di CI/CD ke staging

migrate dev:

- Bisa drop/recreate table jika perlu
- Minta konfirmasi jika destructive
- Reset shadow database
- Jalankan seed

Untuk production: gunakan migrate deploy!

prisma migrate deploy — Untuk Production

Jalankan pending migrations yang belum diapply. Tidak interaktif, tidak destructive.

● ● ● Deployment workflow dengan migrate deploy

```
# — Di package.json scripts —————
{
  "scripts": {
    "build": "prisma generate && next build",
    "postinstall": "prisma generate",
    "db:migrate": "prisma migrate deploy",
  }
}

# — Vercel deployment example —————
# Tambahkan ke Build Command di Vercel:
# prisma generate && prisma migrate deploy && next build

# Atau pakai Vercel Build Output API di vercel.json:
# { "buildCommand": "prisma generate && prisma migrate deploy && next build" }

# — Perbedaan penting —————
# migrate dev:   buat migration BARU + apply
# migrate deploy: HANYA apply migration yang sudah ada

# migrate deploy AMAN karena:
# ✓ Tidak buat migration baru (read-only workflow)
# ✓ Tidak pernah reset/drop database
# ✓ Tidak interaktif (aman di CI/CD)
# ✓ Hanya apply migration yang belum diapply
# ✓ Idempotent — aman dijalankan berkali-kali
```

db push & Prisma Studio

db push untuk prototyping cepat. Prisma Studio untuk visualisasi dan edit data.

```
● ● ● prisma db push — prototyping tanpa migration
```

db push: sync schema ke DB TANPA migration file
Cocok untuk: eksplorasi awal, prototyping

bunx prisma db push

☒ Update database ke schema saat ini
☒ Tidak buat migration file
☒ Data bisa hilang jika ada destructive change

KAPAN pakai db push:
- Proof of concept / rapid prototyping
- Belum siap commit migration
- Learning / exploration

JANGAN pakai db push di project nyata
setelah ada data production!
Gunakan migrate dev untuk versioned changes

Juga berguna untuk setup awal:
bunx prisma db push
← sync schema ke database baru tanpa migration

```
● ● ● Prisma Studio — GUI untuk database
```

Prisma Studio: browser-based GUI untuk DB
bunx prisma studio
→ Buka <http://localhost:5555>

Fitur Prisma Studio:

☒ Browse semua tabel dan data
☒ Filter, sort records
☒ Edit record langsung dari browser
☒ Add/delete records
☒ Explore relasi antar tabel

Sangat berguna untuk:

- Debug data issues
- Verifikasi migration berhasil
- Development: cek data tanpa SQL

Prisma 7 ships Studio baru:
bunx prisma studio
(tidak perlu @prisma/studio separate)

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa perbedaan utama migrate dev vs migrate deploy?

- A) Tidak ada bedanya
- B) migrate dev membuat migration baru untuk development. migrate deploy hanya apply yang sudah ada (untuk production)
- C) migrate deploy lebih cepat dari migrate dev

Q2 — Kapan sebaiknya file migration di-commit ke git?

- A) Tidak perlu — Prisma generate otomatis
- B) Segera setelah dibuat dengan migrate dev — ini adalah database version control
- C) Hanya sebelum deploy ke production

Q3 — Apa risiko utama penggunaan prisma db push di project production?

- A) Terlalu lambat untuk production
- B) Tidak ada migration history — schema berubah tanpa record, bisa destructive tanpa warning
- C) Tidak support semua database

Tugas Mandiri

Praktikkan workflow migration yang benar.

01 Initial Migration

Setelah schema.prisma siap, jalankan: `bunx prisma migrate dev --name initial_schema`. Periksa file SQL yang dibuat di `prisma/migrations/`.

02 Schema Change

Tambahkan field `'published: Boolean @default(false)'` ke model `Post`. Jalankan `migrate dev --name add_published_field`. Verifikasi SQL-nya.

03 Prisma Studio

Jalankan `bunx prisma studio`. Browse tabel. Tambahkan 2-3 user manual via Studio. Verifikasi data ada via `db.user.findMany()` di `page.tsx`.

04 Deploy Script

Update `package.json`: `build script = 'prisma generate && next build'`. Tambahkan `db:migrate = 'prisma migrate deploy'`. Commit semua.



Nama migration harus deskriptif: `add_post_table`, `add_slug_to_posts`, `remove_deprecated_field`.

Yang Sudah Kita Pelajari

- ✓ migrate dev: untuk development — buat file SQL + apply + generate client.
- ✓ migrate deploy: untuk production — hanya apply pending migrations, tidak destructive.
- ✓ Selalu commit migration files ke git — ini adalah database version history.
- ✓ db push: untuk prototyping cepat. Tidak untuk project production dengan data nyata.
- ✓ Prisma Studio: GUI browser untuk browse, edit, dan debug data database.

→ Selanjutnya: **Bab 6 — CRUD: insert, update, delete, query**

